Track 2: Problem Solution using Programming Languages

Key Deliverables for Track 2:

- Working prototype (Hosted link or local deployment instructions)
- Source code repository [GitHub]
- Product screenshots / 2-minute working recorded video demo with key functionalities.
- One-page Business Insights Summary explaining the technical architecture and business value

Problem Statement [P1]
Agentic AI Email Triage & Automated Action Workflow

Brief & Case Scenario:
Professionals suffer from massive inbox overload, spending hours reading and categorizing emails rather than doing deep work. A simple summary is no longer enough. The industry requires an "Agentic AI" system that can read an incoming email, understand the context, and autonomously prepare the logical next step—whether that is drafting a polite decline, extracting meeting coordinates into a calendar invite, or flagging a message from a VIP client as urgent.

Outcome:
A smart web-based email client prototype that acts as a proactive virtual assistant, drastically reducing the time users spend managing their inbox by preparing executable actions for their approval.

Tools:
React/Next.js, Node.js or Python (FastAPI), LLM APIs (OpenAI/Anthropic/Gemini), Mock Email JSON dataset.

Constraints:
The prototype must successfully demonstrate the AI agent preparing at least three distinct types of automated actions (e.g., a drafted reply, a calendar event creation, and a task list extraction).

Key Features:

- **Contextual Thread Summarization:** Condensing long email chains into a 3-bullet-point summary.
- **Automated Action Drafting:** The AI pre-fills a response or form based on the email content.
- **Human-in-the-Loop Approval:** A clean UI where the user can review, edit, and click "Approve & Execute" on the AI's suggested action.
- **Priority Tagging System:** Automatically visually flagging emails as 'Urgent', 'Requires Action', or 'FYI'.

Problem Statement [P2]
OCR-Driven Enterprise Document Summarization Engine

Brief & Case Scenario:
Legal, compliance, and accounting teams are bogged down by processing thousands of scanned, image-based PDFs (contracts, invoices, compliance forms). They need a web portal where users can upload these flat files, extract the raw text using Optical Character Recognition (OCR), and immediately generate a summarized breakdown of the critical clauses, dates, and financial figures.

Outcome:
A high-speed document processing web portal that digitizes legacy scanned documents and provides instant, concise summaries, saving hours of manual data entry and reading time.

Tools:
Java (Spring Boot) or C/C++ (for the backend processing), Tesseract OCR (or similar library), React/Angular for the frontend UI.

Constraints:
The core OCR processing and backend logic must be built using Java or C/C++. The system must successfully parse an image-based PDF or image file (PNG/JPG).

Key Features:

- **Drag-and-Drop Ingestion:** A secure frontend upload zone for scanned documents.
- **Side-by-Side Verification UI:** A split-screen view showing the original scanned image on one side and the extracted OCR text on the other.
- **Key Entity Extraction:** Automatically highlighting critical data points like "Total Amount", "Due Date" or "Signatories."
- **Executive Summary Generation:** Using NLP or a basic LLM integration to output a one-paragraph summary of the extracted text.

Problem Statement [P3]
Gen-AI Dynamic Product Catalog & Marketing Copy Generator

Brief:
E-commerce managers spend hours manually writing product descriptions, SEO tags, and social media copy for hundreds of new inventory items. The challenge is to build a merchant tool where uploading a basic list of technical specifications or a raw product image automatically generates rich, multi-platform marketing collateral.

Outcome:
A merchant productivity portal that accelerates time-to-market by using Generative AI to automatically build compelling product pages and promotional copy from minimal raw input.

Tools:
MERN Stack (MongoDB, Express, React, Node) or Python/Django, Generative AI Text/Vision APIs.

Constraints:
From a single product input (specs or image), the system must generate at least three distinct variations of copy (e.g., an SEO Web Description, a short Instagram caption, and a professional LinkedIn post).

Key Features:

- **Raw Spec Ingestion:** A form to input basic technical details (e.g., weight, material, color).
- **SEO-Optimized Web Description:** AI-generated long-form copy designed for an online storefront.
- **Multi-Platform Marketing Outputs:** Generating tailored copy for different social media channels with appropriate tone and hashtags.
- **Dynamic Tagging:** Automatically generating a list of search/category tags for the product database.

Problem Statement [P4]
Smart Complaint Management System & Ticketing Desk

Brief/Case Scenario:
Customer support teams are overwhelmed by repetitive "Level 1" queries (e.g., "Where is my refund?"). This delays the resolution of complex, high-priority issues. Companies need a system where a conversational AI chatbot (good to have) acts as the first line of defense; if the bot cannot resolve the issue, it seamlessly logs the complaint, categorizes it, and generates a support ticket for a human agent.

Outcome:
An end-to-end support portal that reduces manual ticket sorting and improves resolution time through initial AI deflection and intelligent, automated ticket routing.

Tools:
Python/MERN Stack, NLP/Chatbot Framework (Dialogflow, Rasa, or an LLM API), WebSockets for real-time chat.

Constraints:
The chatbot must autonomously determine the "Severity Level" (Low, Medium, High) of the complaint based on the user's language before generating the ticket.

Key Features:

- **Conversational Interface:** A frontend chat widget for users to explain their issue in natural language.
- **Automated Ticket Generation:** If unresolved, the chat transcript and user details are instantly converted into a formal IT/Support ticket.
- **Severity Scoring & Categorization:** Tags the ticket (e.g., 'Billing', 'Technical') and assigns a priority level.
- **Agent Resolution Dashboard:** A secure backend view where human agents can claim tickets, view the bot transcript, and mark issues as resolved.

Problem Statement [P5]
Multi-Tier Corporate Procurement & Expense Approval Workflow

Brief:
Approving corporate expenses and procurement requests often involves messy email chains, lost digital receipts, and delayed payments. Mid-sized companies need a centralized, rule-based web portal where employees submit expenses, and the system automatically routes them to the correct managers based on the department and the dollar amount requested.

Outcome:
A streamlined, secure financial workflow portal that enforces corporate spending policies, tracks approval bottlenecks, and fully digitizes the expense claim process.

Tools:
React/Vue, Node.js/Express or Java Spring Boot, PostgreSQL/MongoDB.

Constraints:
Must implement strict Role-Based Access Control (RBAC) with at least three roles (Employee, Manager, Finance Admin) and dynamic routing logic (e.g., expenses under $100 are auto-approved, expenses over $1,000 require two levels of human approval).

Key Features:

- **Digital Claim Form:** An interface for employees to input expense details and upload digital receipt files.
- **Rule-Based Routing Engine:** Backend logic that reads the expense amount and department to determine who needs to approve it next.
- **Manager Review Dashboard:** A specific view for managers to see pending requests, download receipts, and click "Approve" or "Reject with Comments."
- **Audit Trail & Status Log:** A timeline view for the employee to see exactly where their request is sitting in the approval chain.